

Place the link after the list of existing topics. Figure 19-1 shows the resulting form; try using the form to add a few new topics of your own.



Figure 19-1: The page for adding a new topic

Adding New Entries

Now that the user can add a new topic, they'll want to add new entries too. We'll again define a URL, write a view function and a template, and link to the page. But first, we'll add another class to *forms.py*.

The Entry ModelForm

We need to create a form associated with the Entry model, but this time, with a bit more customization than *TopicForm*:

```
forms.py  from django import forms

          from .models import Topic, Entry

          class TopicForm(forms.ModelForm):
              --snip--

          class EntryForm(forms.ModelForm):
              class Meta:
                  model = Entry
                  fields = ['text']
                  labels = {'text': ''}
                  widgets = {'text': forms.Textarea(attrs={'cols': 80})}
```

We update the import statement to include *Entry* as well as *Topic*. We make a new class called *EntryForm* that inherits from *forms.ModelForm*. The *EntryForm* class has a nested *Meta* class listing the model it's based on, and the field to include in the form. We again give the field 'text' a blank label ❶.

For *EntryForm*, we include the *widgets* attribute ❷. A *widget* is an HTML form element, such as a single-line text box, multiline text area, or drop-down list. By including the *widgets* attribute, you can override Django's